UNIVERSIDADE POLITÉCNICA DO CÁVADO E DO AVE

ESCOLA SUPERIOR DE TECNOLOGIA UPCA

UPCA

Universidade Politécnica do Cávado e Ave

ETL

Análise e Otimização de Dados para sugestões de Alojamento Local

Curso

Licenciatura em Engenharia de

Sistemas Informáticos

Unidade Curricular

Integração de Sistemas Informáticos

Realizado por:

Fábio Ferreira nº 29365

Docente:

Luís Gonzaga Martins Ferreira

Data: 22/10/2026

Índice

1. Enquadramento 4

2. Problema 4

3. Estratégia 5

4. Processo ETL dos dados 7

4.1 - Extração 7

4.2 - Transformação 9

4.3 - Metanodes 19

4.3.1 – Extração de Dados 19

4.3.2 – Nodos de Transformação de dados 20

4.3.3 – Nodos de Transformação de dados 2 – Agregações 20

4.3.4 – Nodos de Exportação 21

4.4 – Exportação 21

5 - Vídeo com demonstração 27

6 - Conclusão 27

7 - Bibliografia 28

Índice de Figuras

Figura 1 - Conteúdo do ficheiro XML 5
Figura 2 - Extração dos ficheiros CSV 7
Figura 3 - Extração dos ficheiros XML 8
Figura 4 - Informações dos Dados após o Joiner 8
Figura 5 - Informações dos Dados após o Column Filter 9
Figura 6 - Configuração da Chamada à API 10
Figura 7 - Transformação dos dados até obter os preços em euros 10
Figura 8 - Transformação dos dados de preços dos alojamentos 11
Figura 9 - Configuração do regex para a coluna host_name 12
Figura 10 - Column Filter aplicado antes de anonimizar 14
Figura 11 - Mapa dos nodos da agregação de dados 15
Figura 12 - Dados agrupados do nodo GroupBy 15
Figura 13 - Configuração da Rule Engine 16
Figura 14 - Ligações finais de nodos para ter os dados completos por freguesia 17
Figura 15 - Parte final da Transformação, exportação para ficheiros de output ... 18
Figura 16 - Fluxo organizado por metanodes 19
Figura 17 - Metanode 1 - Extração - Nodos encapsulados 19
Figura 18 - Metanode 2 – Transformação - Nodos encapsulados 20
Figura 19 - Metanode 3 – Transformação Agregação - Nodos encapsulados 20
Figura 20 - Metanode 4 – Exportação - Nodos encapsulados 21
Figura 21 - Ficheiros output 21
Figura 22 - MongoDB 22
Figura 23 - Docker a correr o container Node-RED 23
Figura 24 - Fluxo geral dos nodos 24
Figura 25 - Exemplo do código de um nodo function 24
Figura 26 - Exposição dos dados no mapa interativo 25
Figura 27 - Fluxo geral para atualização dos dados API 26

1. Enquadramento

Este trabalho foi feito no âmbito da disciplina de Integração de Sistemas Informáticos, no curso de Engenharia de Sistemas Informáticos em regime Pós Laboral.

2. Problema

O setor do turismo enfrenta desafios contínuos na análise de dados geográficos, de custos e de qualidade de alojamento. O problema que se pretende resolver consiste em recolher e transformar grandes quantidades de dados relativos à oferta de alojamento local nas regiões de Lisboa e do Porto.

O objetivo final é processar e transformar estes dados para criar um sistema de apoio à decisão na base da recomendação e de classificações, capaz de identificar as melhores áreas e alojamentos com base numa relação entre o preço, localização e avaliações de qualidade, apresentando posteriormente os resultados de forma dinâmica num ambiente interativo com recurso a um mapa.

3. Estratégia

Para dar cumprimento aos requisitos de diversidade de fontes e formatos exigidos, usou-se uma estratégia de integração em três fases principais, *ETL*, **Extract-Transform-Load**:

Fase 1 **Extração** – (Extraction): Foram recolhidos dados a partir de fontes de vários tipos, englobando ficheiros estruturados em formato CSV que foram dados de listagens do *Inside Airbnb* para Lisboa e o Porto e ficheiros em formato XML sendo esses feitos de tabelas de regras customizadas com as freguesias e taxas turísticas municipais, usadas para complementar e uniformizar algumas colunas de dados do CSV.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<tabela_regras>
    <registo>
        <nome_csv>Cedofeita, Ildefonso, Sé, Miragaia, Nicolau, Vitória</nome_csv>
        <nome_corrigido>Centro Histórico</nome_corrigido>
        <concelho>Porto</concelho>
        <taxa_turistica>3.00</taxa_turistica>
    </registo>
    <registo>
        <nome_csv>Bonfim</nome_csv>
        <nome_corrigido>Bonfim</nome_corrigido>
        <concelho>Porto</concelho>
        <taxa_turistica>3.00</taxa_turistica>
    </registo>
    <registo>
        <nome_csv>Santa Marinha e São Pedro da Afurada</nome_csv>
        <nome_corrigido>Santa Marinha</nome_corrigido>
        <concelho>Vila Nova de Gaia</concelho>
        <taxa_turistica>2.50</taxa_turistica>
    </registo>
    <registo>
        <nome_csv>Lordelo do Ouro e Massarelos</nome_csv>
        <nome_corrigido>Lordelo do Ouro</nome_corrigido>
        <concelho>Porto</concelho>
        <taxa_turistica>3.00</taxa_turistica>
    </registo>
    <registo>
        <nome_csv>Paranhos</nome_csv>
        <nome_corrigido>Paranhos</nome_corrigido>
        <concelho>Porto</concelho>
        <taxa_turistica>3.00</taxa_turistica>
    </registo>
```

Figura 1 - Conteúdo do ficheiro XML

Fase 2 **Transformação** – (Transformation): Operações executadas na plataforma KNIME, com o objetivo de conseguir a concatenação, filtragem e limpeza de valores desformatados em datasets de alojamentos regionais e normalização de textos, cruzamento de dados para o enriquecimento dos dados de alojamento com as respetivas taxas turísticas e outras sugestões com base em dados requisitados por HTTP vindos de APIs públicas.

Fase 3 **Carga** – (Load): Após as exportações dos dados, são feitas inserções dos mesmos em bases de dados, como a MongoDB, com a finalidade de persistência dos dados, e integração com outras plataformas como a Node-RED, com o uso de exposição e interação com estes dados.

4. Processo ETL dos dados

4.1 - Extração

Começando da esquerda para a direita, temos os dois nodos ***CSV Reader*** que fazem a leitura dos ficheiros CSV que eu forneci, Listings_Porto.csv e Listings_Lisboa.csv, esses ficheiros continham informações brutas tiradas do Inside Airbnb, com informações sobre taxas, preços, localidades, etc, com vários dados desformatados e em falta. Após isso coloquei o nodo ***Concatenate*** para juntar as tabelas de ambos os CSV de Lisboa e Porto numa só para continuar as operações nos dados.

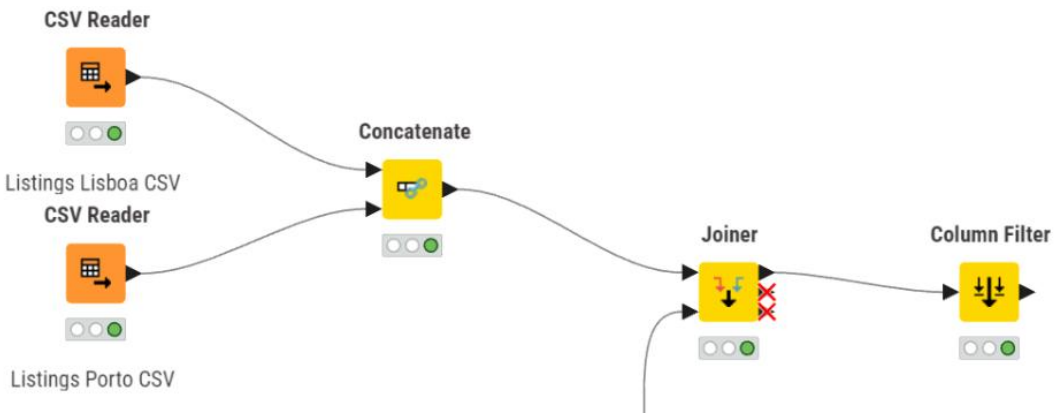


Figura 2 - Extração dos ficheiros CSV

Em seguida, parecido com as operações em cima, adicionei nodos ***XML Reader*** para ler os ficheiros XML (fig. 3). Estes XML continham informações retificadas sobre as colunas de interesse dos listings.csv.

O objetivo dos nodos era concatenar os dados dos XML que estão em *tags* (fig. 1), para depois com o nodo ***XPath*** transformar tudo em tabelas, prontas para ligar ao ***Joiner*** (fig. 3) que em seguida fez a função de fundir os dados em tabelas dos XML com as tabelas dos CSV.

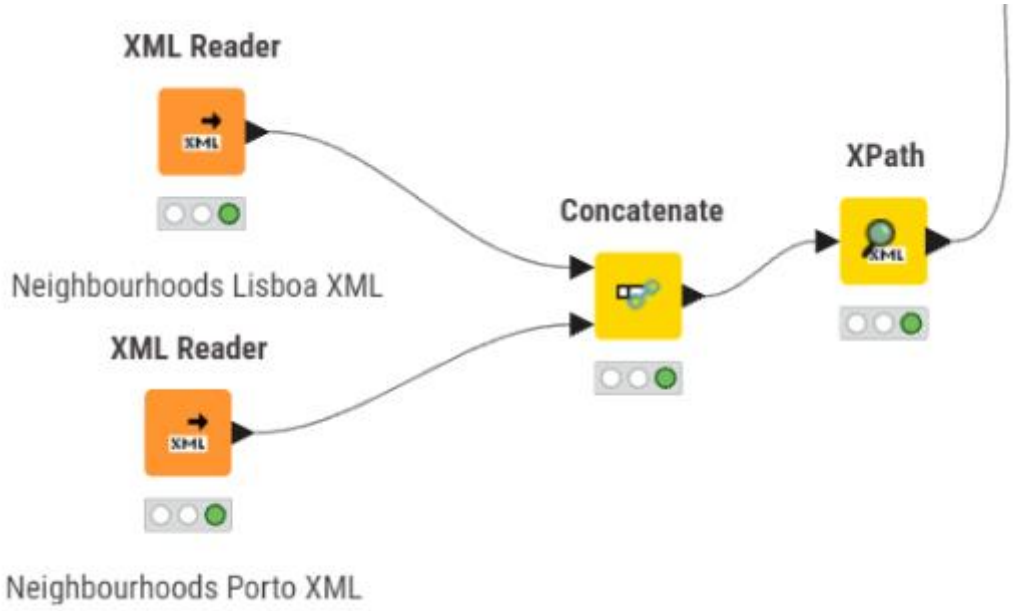


Figura 3 - Extração dos ficheiros XML

Seguido do nodo *Joiner*, que já tinha juntos os dados todos que eu iria precisar para o meu objetivo, faltava só "limpar" a quantidade de dados que eu queria usar, já que o Joiner no momento possuía tremendas quantidades de Linhas e Colunas de dados (fig. 4), sendo assim preciso colocar um ***Column Filter*** que me deixou filtrar as colunas que eu queria usar, baixando a quantidade de colunas de 94 para só **14.** (fig. 5)

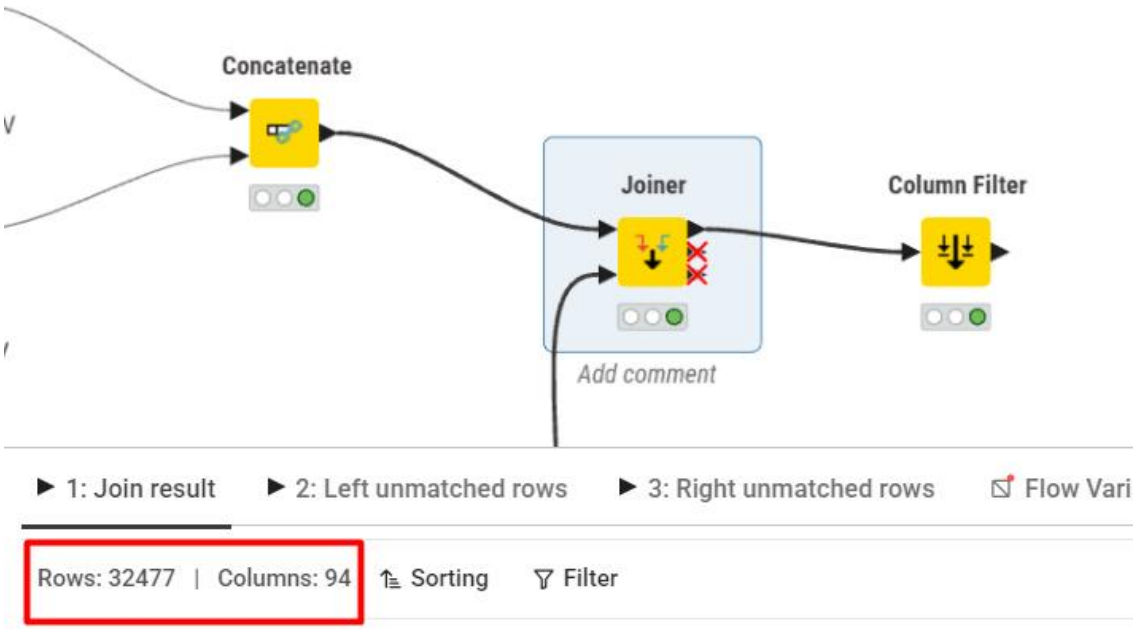


Figura 4 - Informações dos Dados após o Joiner

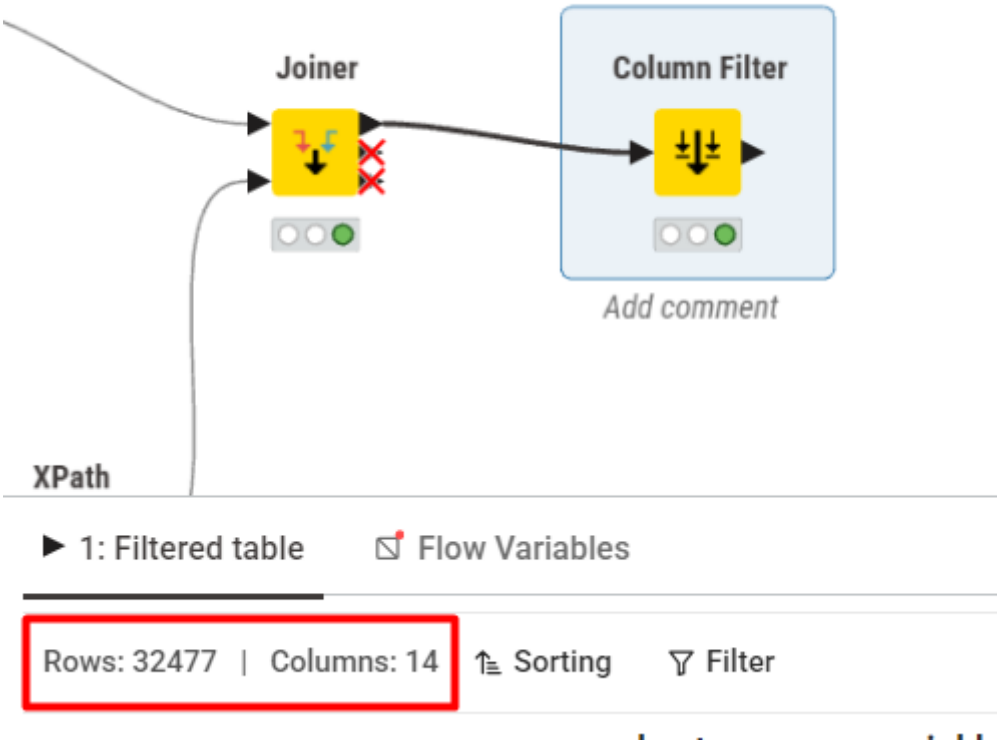


Figura 5 - Informações dos Dados após o Column Filter

4.2 - Transformação

Agora tendo organizado a quantidade certa de dados, começa a parte em que se tem de transformar os mesmos, já que muitos estão desformatados, e até nulos/em falta e precisam de ser uniformizados.

Comecei por fazer uma chamada à API da *Frankfurter,* que dá dados em JSON com informações de câmbio de moedas, para conseguir converter os valores dos preços na tabela principal, que vinham em dólar, sendo o meu objetivo o resultado em euros.

Esta chamada foi feita com um node *GET Request*, que foi configurado para fazer uma chamada *HTTP* à *API* com um **GET** para conseguir a informação do câmbio, sendo esta obtida no formato JSON, extraída com o nodo *JSON Path*, que em diante usei para criar uma **"variável local"** com o nodo *Table Row to Variable.*



Figura 6 - Configuração da Chamada à API

Continuando de onde deixei na extração, com a informação toda extraída nas 14 colunas, continuei a transformar essa informação com 3 nodos: *Missing Value* para remover linhas de informação nas colunas que não tinham informação, *String Manipulation* para remover o **$** dos valores dos preços que vinham do CSV, *String to Number* para transformar os valores dos preços de **string** para **double**, assim aptos para as contas com a variável obtida da API para o câmbio de Dólar para Euro. Finalmente aplicando o nodo *Math Formula juntamente com a variável obtida* converti os preços para euro (fig 7).

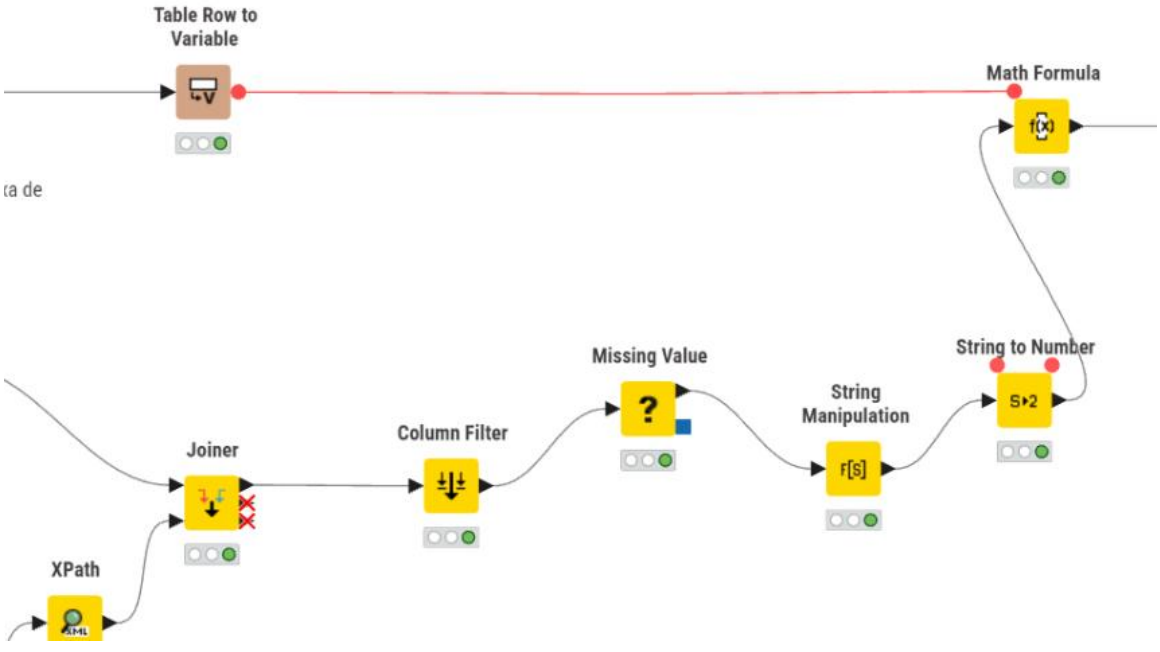


Figura 7 - Transformação dos dados até obter os preços em euros

Tendo os valores dos preços agora uniformizados em euros na coluna *price*, peguei na coluna de *taxa_turistica* de cada concelho, e com outro nodo *Math Formula*, adicionei a coluna *preco_noite*, que continha o valor real por noite de cada alojamento.

Seguido disso, como eu planeei calcular médias mais à frente, eu meti um nodo *Row Filter* que serviu para filtrar só os alojamentos com preços inferiores a 1500 euros, já que existiam anúncios de alojamentos que chegavam aos 10 milhares de euros, eram poucos, mas iriam deformar a média obtida (fig. 8). Com outro nodo *String Manipulation,* apliquei uma fórmula Regex para substituir a coluna host_name com o propósito de anonimizar os hosts/donos de cada anúncio, para proteger a sua privacidade, por exemplo: Rui->R** (fig. 9).

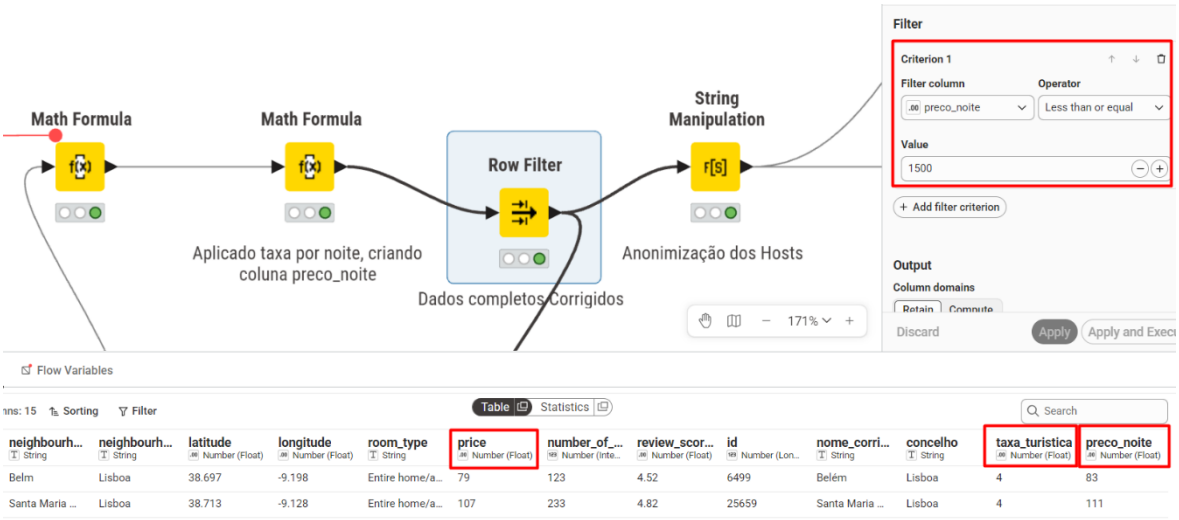


Figura 8 - Transformação dos dados de preços dos alojamentos

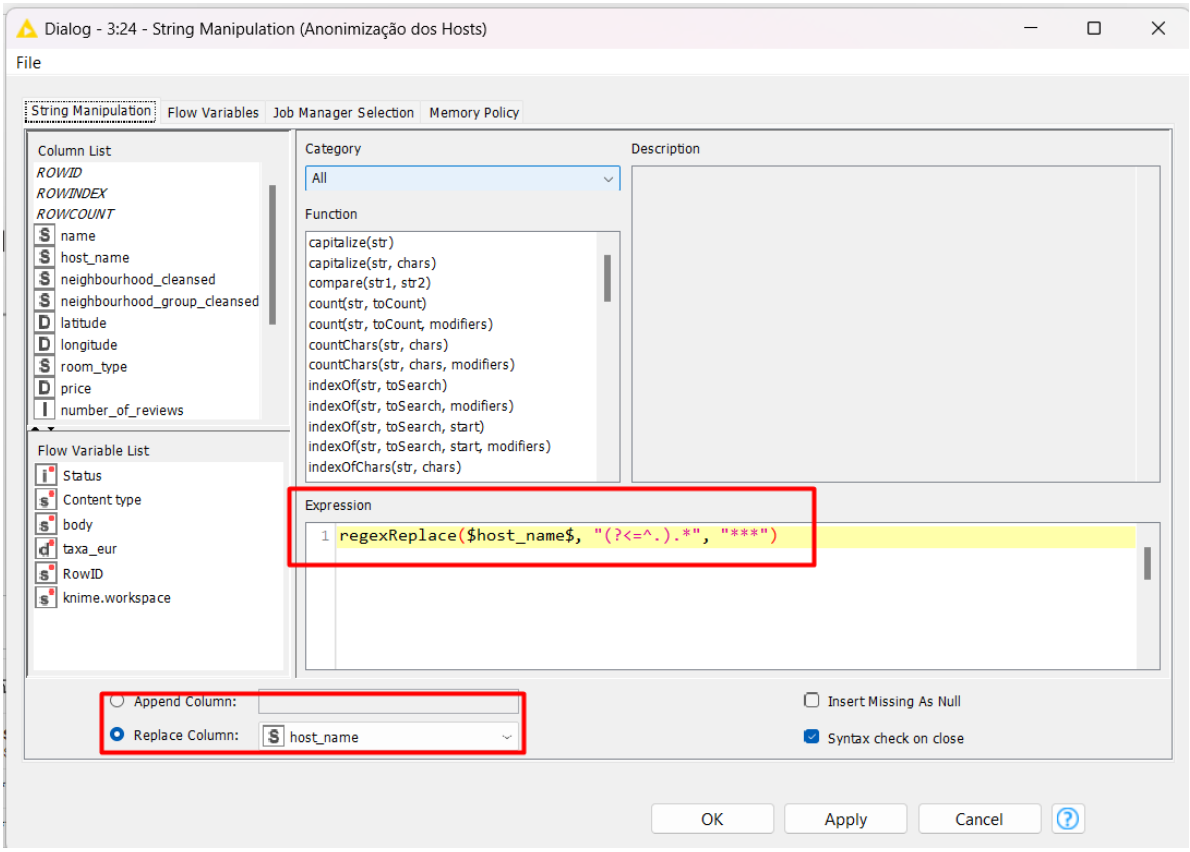


Figura 9 - Configuração do regex para a coluna host_name

Com isto, chegando ao nodo *String Manipulation,* tinha todos os dados que eu iria precisar para a parte de **Load,** este nodo tinha as seguintes colunas:

Nome da Coluna	Tipo	Descrição
name	String	Nome do alojamento.
host_name	String	Nome do dono do alojamento.
neighbourhood_cleansed	String	Localidade do alojamento, originária dos dados brutos, mantida para comparação

Nome da Coluna	Tipo	Descrição
		com a coluna nome_corrigido.
neighbourhood_group_cleansed	String	Concelho do alojamento, originária dos dados brutos, mantida para comparação com a coluna concelho.
latitude	Number (Float)	Coordenada geográfica para representar o alojamento no mapa.
longitude	Number (Float)	Coordenada geográfica para representar o alojamento no mapa.
room_type	String	Tipo de alojamento (ex: casa inteira, quarto privado).
price	Number (Float)	Preço bruto do alojamento em dólares/euros.
number_of_reviews	Number (Integer)	Número total de avaliações recebidas pelo alojamento.
review_scores_rating	Number (Float)	Classificação média das avaliações do alojamento.

Nome da Coluna	Tipo	Descrição
id	Number (Long)	Identificador único de cada alojamento.
nome_corrigido	String	Nome retificado da freguesia, processado através do XML.
concelho	String	Nome retificado do concelho, processado através do XML.
taxa_turistica	Number (Float)	Taxa municipal a pagar por noite (varia consoante a localidade).
preco_noite	Number (Float)	Preço final total por noite, já convertido para euros e com taxa aplicada.

Mas antes de ter tudo anonimizado, apliquei outro nodo **column filter** para retirar **neighbourhood_cleansed e neighbourhood_group_cleansed,** agora ficando só com as 13 colunas com os dados necessários.

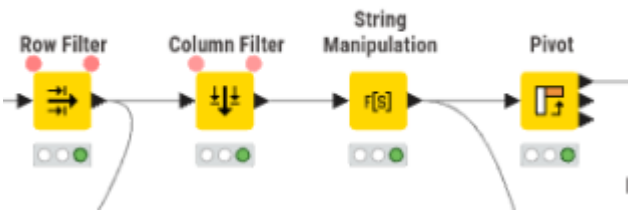


Figura 10 - Column Filter aplicado antes de anonimizar

Analisando a figura 8, o RowFilter com a informação importante toda, tem uma ligação para baixo, que leva a esta série de nodos que têm a finalidade de criar outro set de dados que continham médias de preços, *reviews* e informações que ajudam o cliente a fazer uma decisão sobre os alojamentos (fig. 11).

O nodo *GroupBy,* serviu para iniciar esta agregação por juntar os dados que têm a mesma freguesia, e com esses valores fazer a média de quatro colunas de relevo: preco_noite, review_scores_rating, latitude, longitude, tal como a contagem de Ids para saber quantos alojamentos havia por freguesia (fig. 12).

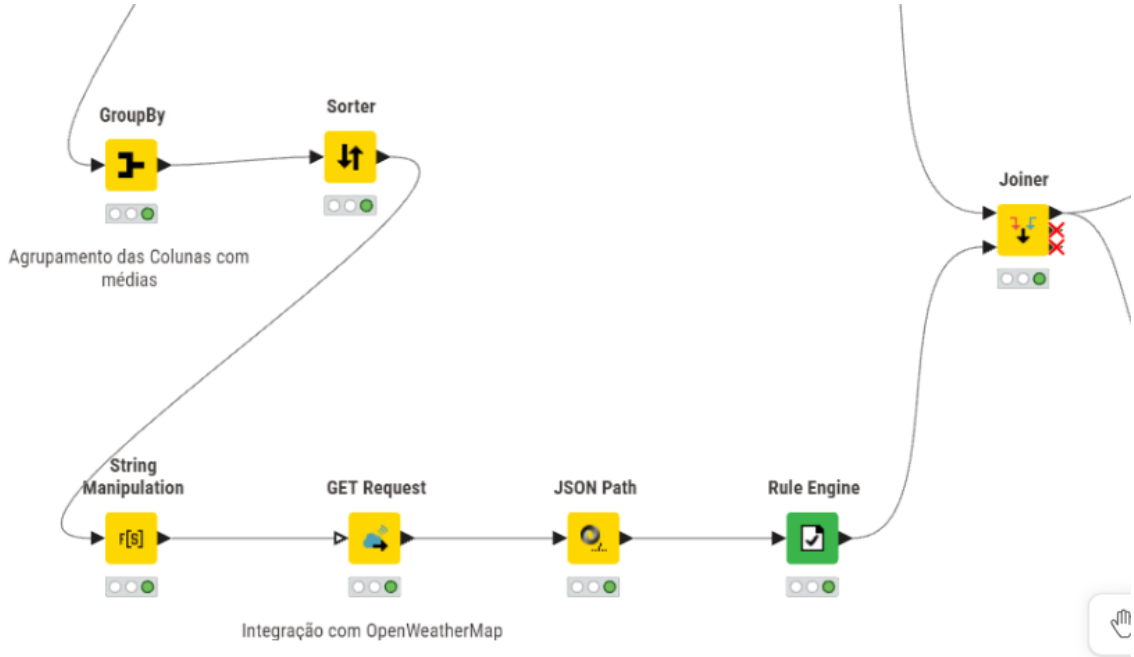
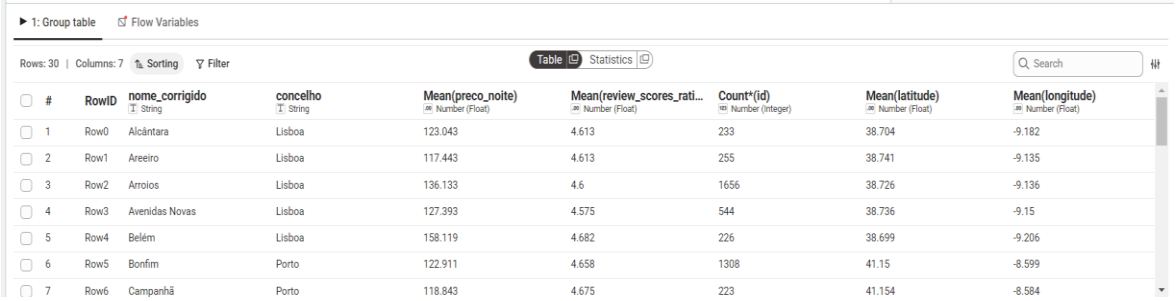


Figura 11 - Mapa dos nodos da agregação de dados



1: Group table | Flow Variables

Rows: 30 | Columns: 7 | Sorting | Filter | Table | Statistics | Search

#	RowID	nome_corrigido (String)	concelho (String)	Mean(preco_noite) (Number (Float))	Mean(review_scores_rati... (Number (Float))	Count*(id) (Number (Integer))	Mean(latitude) (Number (Float))	Mean(longitude) (Number (Float))
1	Row0	Alcântara	Lisboa	123.043	4.613	233	38.704	-9.182
2	Row1	Areeiro	Lisboa	117.443	4.613	255	38.741	-9.135
3	Row2	Arroios	Lisboa	136.133	4.6	1656	38.726	-9.136
4	Row3	Avenidas Novas	Lisboa	127.393	4.575	544	38.736	-9.15
5	Row4	Belém	Lisboa	158.119	4.682	226	38.699	-9.206
6	Row5	Bonfim	Porto	122.911	4.658	1308	41.15	-8.599
7	Row6	Campanhã	Porto	118.843	4.675	223	41.154	-8.584

Figura 12 - Dados agrupados do nodo GroupBy

Depois do nodo *GroupBy* usei um nodo *sorter* para apresentar as médias de forma ascendente baseadas na coluna do *preco_noite.*

Seguido disso, no nodo *String Manipulation* foi onde comecei por contactar a segunda API pública do OpenWeatherMap, a minha ideia foi conseguir os dados da meteorologia em cada freguesia para depois apresentar mais sugestões por alojamento. O *String Manipulation* adiciona uma coluna *url_meteo* aos dados para o nodo *seguinte, GET Request,* conseguir o url da *API* em cada freguesia e enviar uma requisição HTTP com GET.

Depois de feita a requisição, se chegar com código 200, o nodo *JSON Path*, traduz de JSON para coluna a informação relevante da API que entra num nodo especial *Rule Engine* que faz o seguinte (fig. 13):

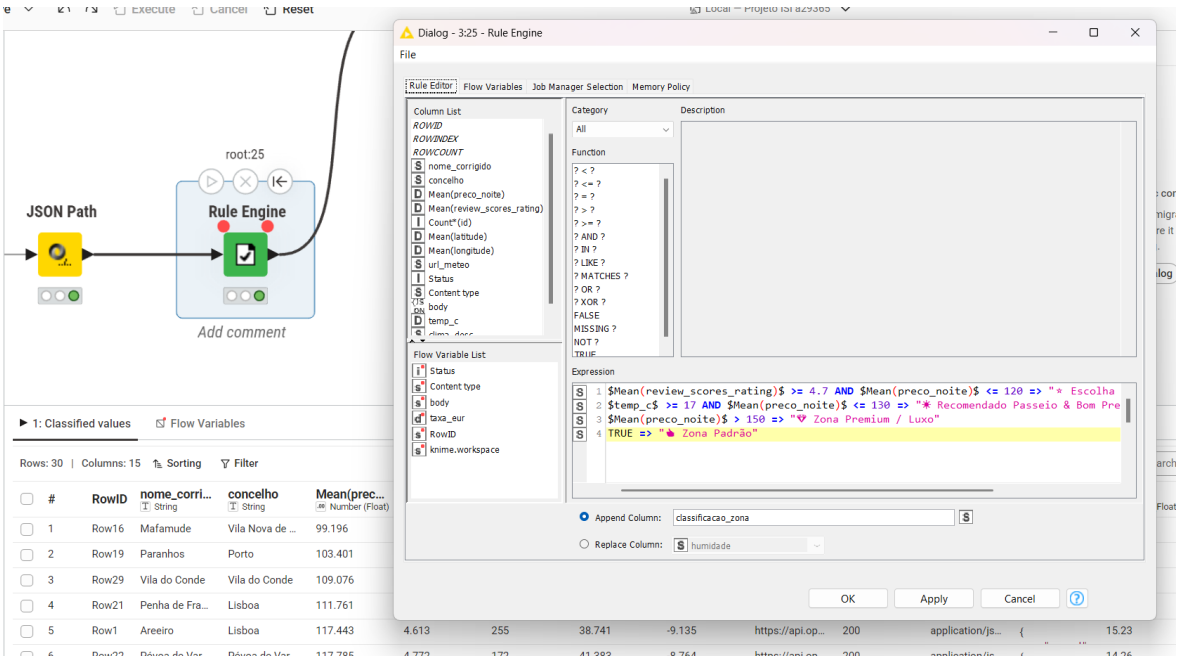


Figura 13 - Configuração da Rule Engine

Definiu-se um *conjunto de regras* baseadas nas médias de reviews e preços, tal como a temperatura obtida da API que cria uma coluna de dados nova para cada freguesia a dizer sugestões do estado do alojamento e da sua área ao redor.

Após ter tudo direito com a *Rule Engine*, este set de dados estava pronto para enviar para um nodo *Joiner* que se liga ao resto dos nodos da parte de cima, continuando da anonimização dos hosts, na tabela com o set inteiro das informações de todos os alojamentos não agrupados por freguesia.

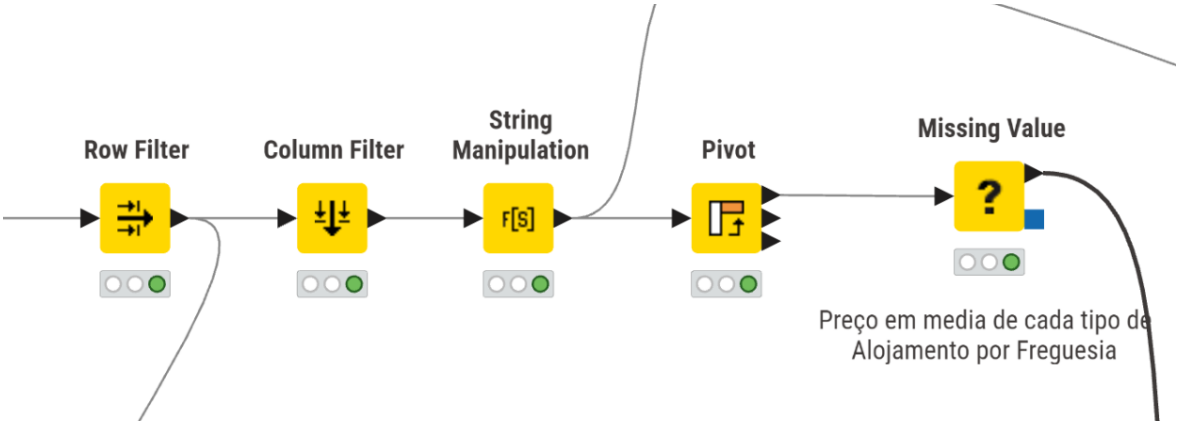


Figura 14 - Ligações finais de nodos para ter os dados completos por freguesia

Este fluxo foi feito com a filtração mais eficiente em mente, ou seja, depois de anonimizar os hosts com o nodo *String Manipulation*, com o nodo *Pivot* (fig. 14) agrupei os dados pelas 30 freguesias nas linhas e transpus a coluna *room_type* para colunas, calculando as médias do *preco_noite* para cada tipo de alojamento (casa inteira, quarto privado, quarto partilhado e quarto de hotel). Em seguida, utilizei um nodo *Missing Value* (fig. 14) para substituir por 0 os campos nulos das freguesias onde não existia oferta de determinado tipo de quarto, enviando finalmente o resultado para o *Joiner* final.

Finalmente, para escrever os sets de dados em ficheiros de output, para depois inserir na base de dados, na 3ª parte deste processo ETL, criaram-se 3 nodos de escrita: 2 nodos *CSV Writer*, um para gravar os dados do dataset completo depois da anonimização após o *String Manipulation antes do Pivot,* outro *CSV Writer para gravar a tabela de dados após o nodo Joiner* que continha todos os dados das médias dos preços por freguesia, com as informações e sugestões da meteorologia, e *um Table to JSON* que tinha a função de transformar as tabelas no formato JSON, para entrarem no último nodo *JSON Writer,* que colocará os dados do *Joiner* em formato JSON para dar output em um ficheiro .json para usar no MongoDB e *NODE-RED* no próximo passo (fig. 15).

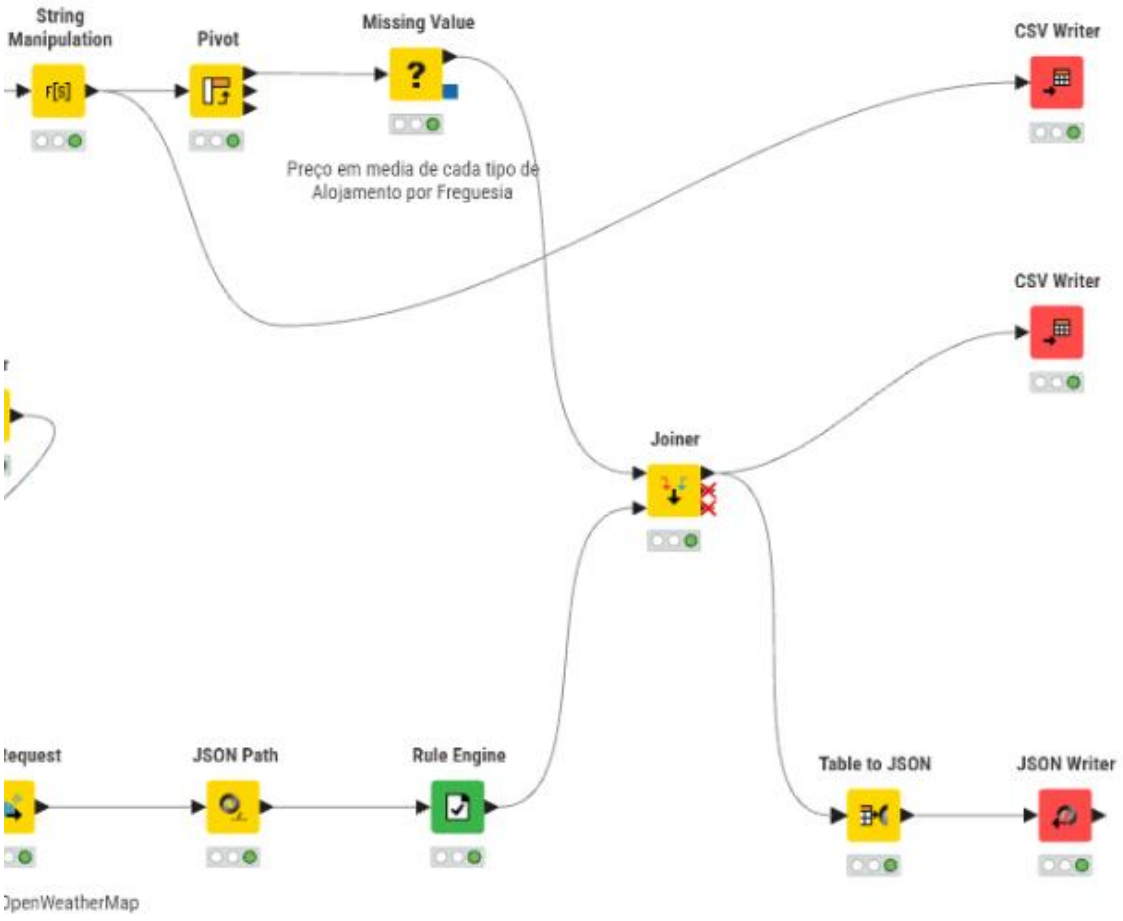


Figura 15 - Parte final da Transformação, exportação para ficheiros de output

4.3 - Metanodes

Para estruturar e orquestrar a execução completa do pipeline de ETL no KNIME, os nodes foram encapsulados em quatro *Metanodes* sequenciais que ilustram a ordem de Extração, Transformação e Exportação dos dados de forma organizada e que mantém os resultados originais (fig. 16):

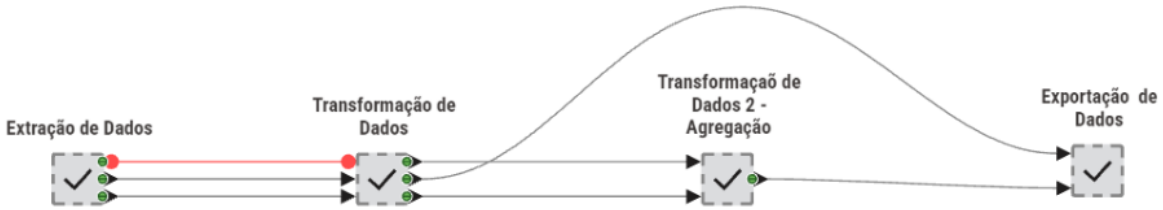


Figura 16 - Fluxo organizado por metanodes

4.3.1 – Extração de Dados

Este *metanode* encapsula a parte de extração de dados, que continha os nodos de *CSV Reader, Get Request, JSON Path, Table Row to Variable, Concatenate, XPath (fig. 17)*

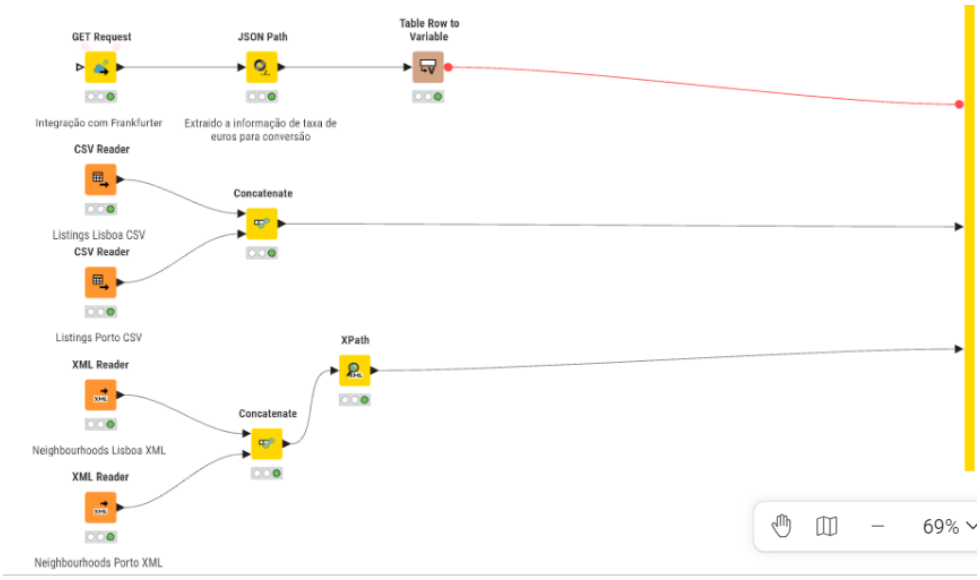


Figura 17 - Metanode 1 - Extração - Nodos encapsulados

4.3.2 – Nodos de Transformação de dados

Este *metanode* encapsula a parte de transformação de dados, que continha os nodos de *Joiner, Column Filter, Missing Value, String Manipulation, String to Number, os dois Math Formula, o Row Filter, o segundo Column Filter, o String Manipulation (anonimização), o Pivot e o Missing Value* (fig. 18)

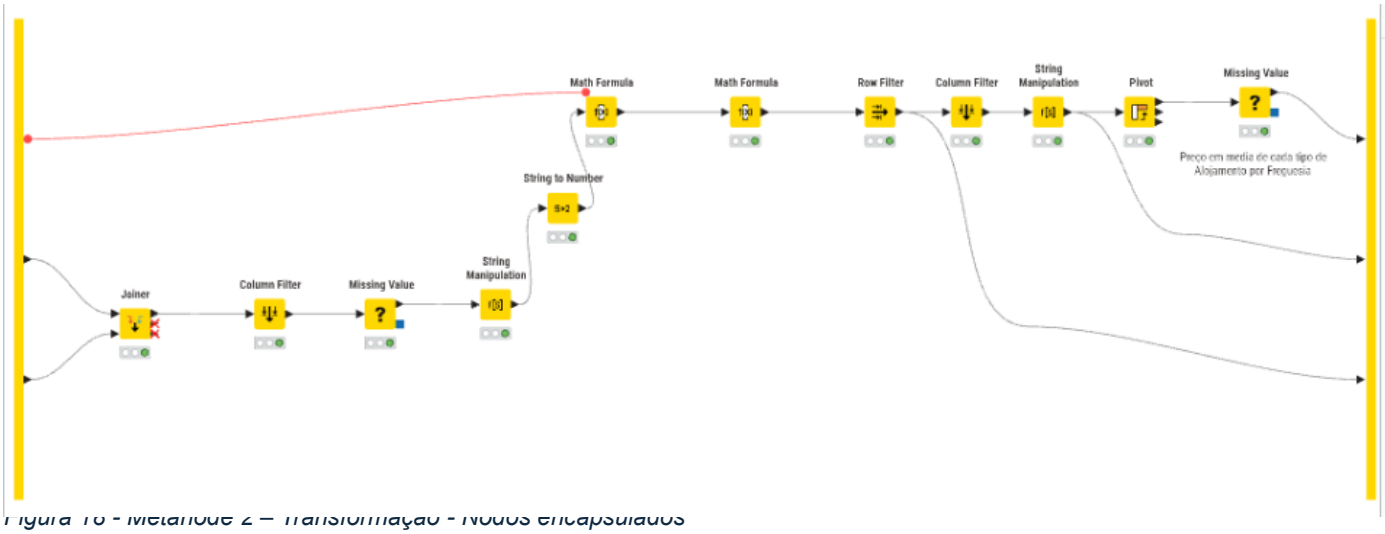


Figura 18 - Metanode 2 – Transformação - Nodos encapsulados

4.3.3 – Nodos de Transformação de dados 2 – Agregações

Este *metanode* encapsula a segunda parte de transformação de dados, que consiste na parte de agregação das freguesias para conseguir as médias de reviews, preços, etc tal como a informação da meteorologia, contém os nodos de *GroupBy, Sorter, String Manipulation (coluna url_meteo), GET Request, JSON Path, Rule Engine e o Joiner final* (fig. 19)

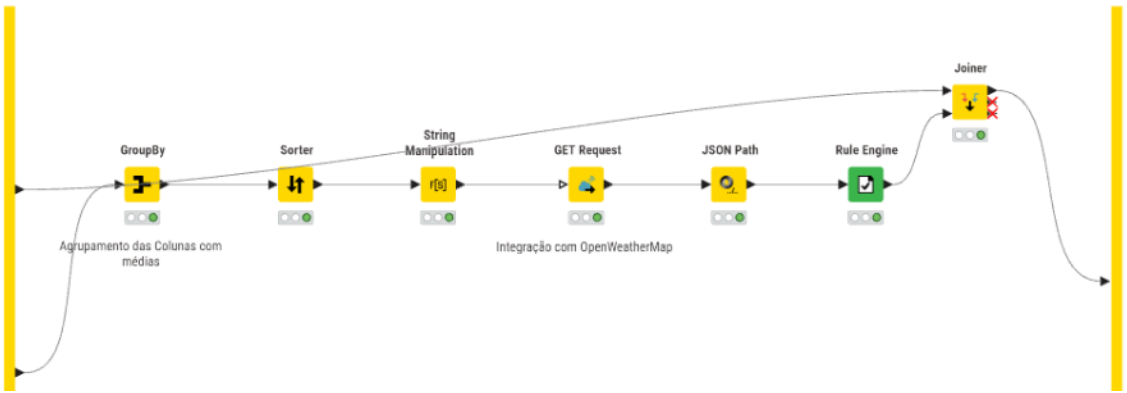


Figura 19 - Metanode 3 – Transformação Agregação - Nodos encapsulados

4.3.4 – Nodos de Exportação

Este *metanode* encapsula a parte de exportação de dados, que consiste em escrever os dados para ficheiros CSV e JSON, contém os dois nodos *CSV Writer, o Table to JSON e o JSON Writer* (fig. 20):

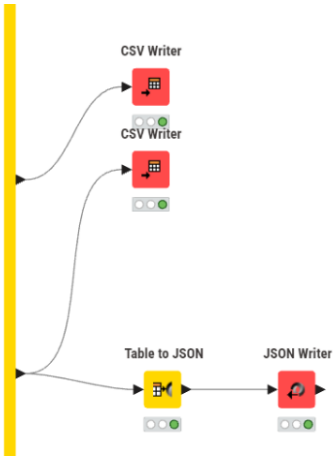


Figura 20 - Metanode 4 – Exportação - Nodos encapsulados

4.4 – Exportação

Para esta última parte do processo ETL, peguei nos ficheiros de output transformados vindos do KNIME: *alojamentos_limpos.csv, freguesias_dados.json, resumo_freguesias.csv,* sendo resumo_freguesias.csv igual ao freguesias_dados.json em termos de informação, resumo_freguesias.csv acabou por servir para fins de teste e para mostrar que deu para escrever o mesmo dataset em ambos csv e JSON (fig. 21).

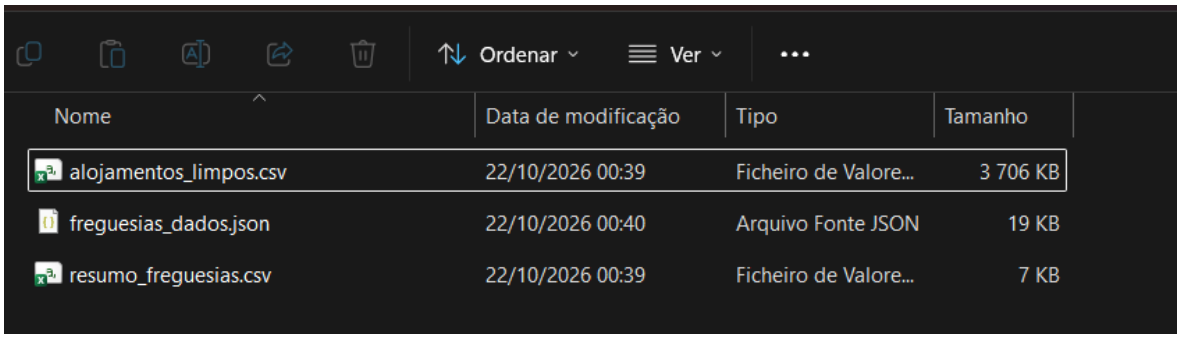


Figura 21 - Ficheiros output

Continuando, acedi ao meu cluster da *MongoDB* na *cloud*, sendo essa a base de dados que eu escolhi para este trabalho. Escolhi a MongoDB porque é não relacional, ou seja, não tenho de definir tabelas rígidas antes de inserir dados, e é perfeita para lidar com ficheiros JSON e CSV, que é o caso deste trabalho.

Para começar, criei uma ***base de dados*** chamada *isi_alojamento*, com duas ***collections*** (equivalente a uma tabela numa base de dados relacional)***: alojamentos, freguesias***, sendo que alojamentos contém guardado os dados do ficheiro alojamentos_limpos.csv, transformado em json na BD, e freguesias contém o ficheiro freguesias_dados.json (fig. 22).

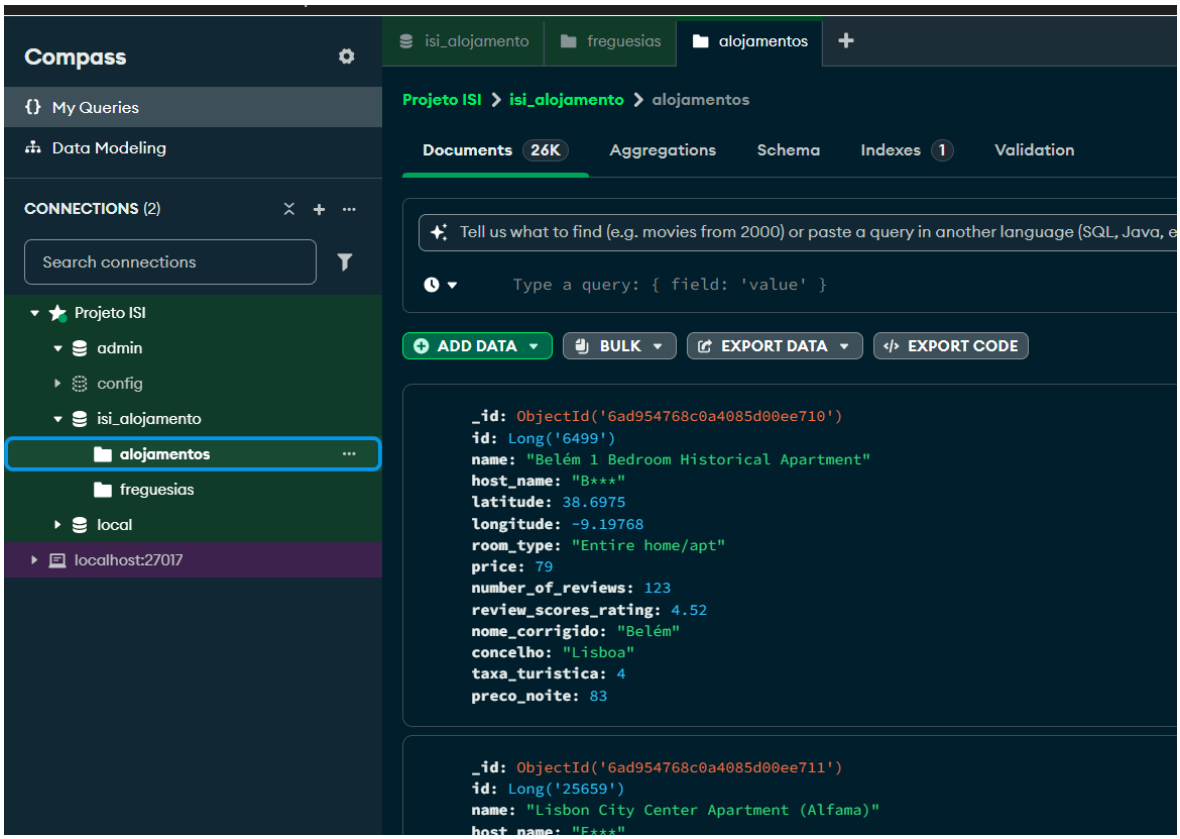


Figura 22 - MongoDB

Com os dados persistidos na base de dados, avançou-se para a sua integração e visualização no Node-RED. A plataforma foi levantada com o uso de Docker, uma tecnologia de contentorização.

Através de um ficheiro de orquestração *docker-compose.yml*, o contentor do Node-RED foi levantado de forma automática. Este ficheiro define também a criação de um volume permanente, o que garante que as configurações e os fluxos desenvolvidos ficam guardados numa pasta local, facilitando a exportação e também a portabilidade do projeto para outros sistemas (fig. 23).

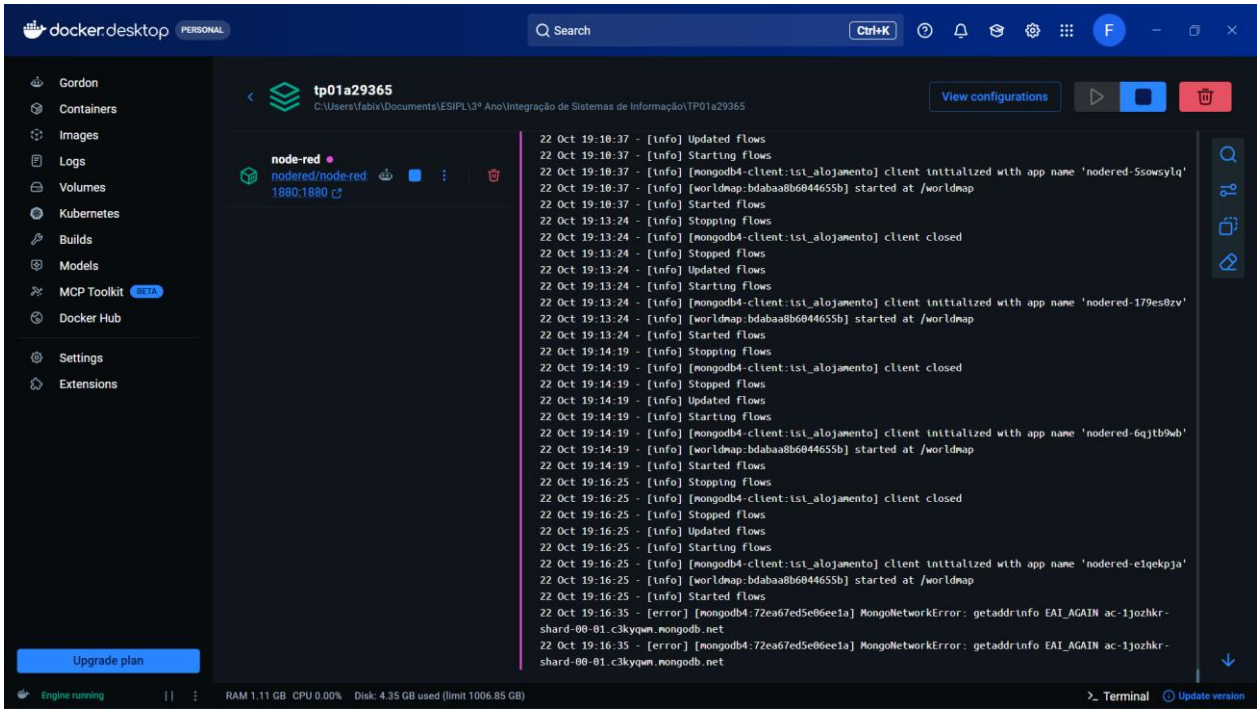


Figura 23 - Docker a correr o container Node-RED

À semelhança do KNIME, o fluxo de dados no Node-RED é processado da esquerda para a direita através de nós com funções específicas. O processo inicia-se com um nó *inject*, que atua como o gatilho do fluxo. Este nó envia um objeto JSON vazio ({}) para acionar os nós seguintes, instruindo as ligações ao MongoDB a executar operações de pesquisa (find) que extraem os dados armazenados nas coleções.

Após obter os dados através do nodo *find* do ramo de cima, que serve para obter os dados dos alojamentos, a *function1* tem código *JavaScript* que serve para selecionar 10 alojamentos em **todas as freguesias** para expor certos dados**,** enquanto no ramo de baixo, o *find* devolve os dados das freguesias, que em seguida são processados na *function2* que contém código para selecionar todas as freguesias e expor certos dados sobre as mesmas (fig. 25). Os nodos *Debug* foram muito úteis para arranjar problemas com conexão na base de dados que foram eventualmente surgindo na aprendizagem desta plataforma (fig. 24).

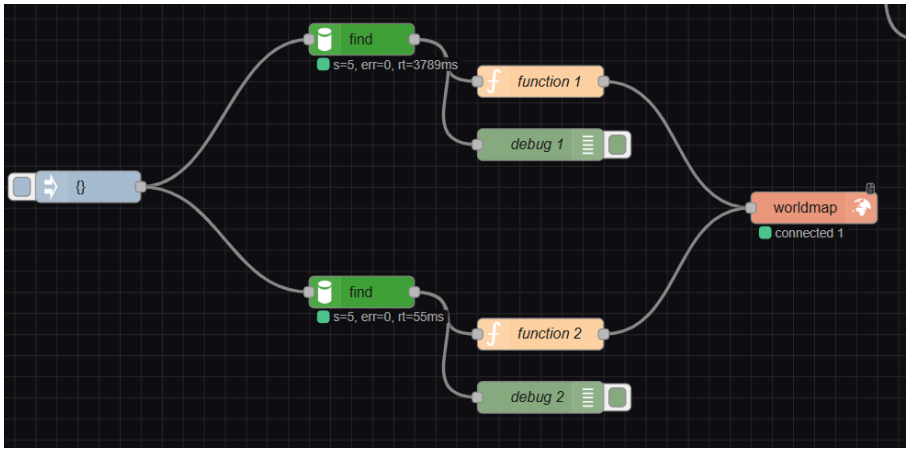


Figura 24 - Fluxo geral dos nodos

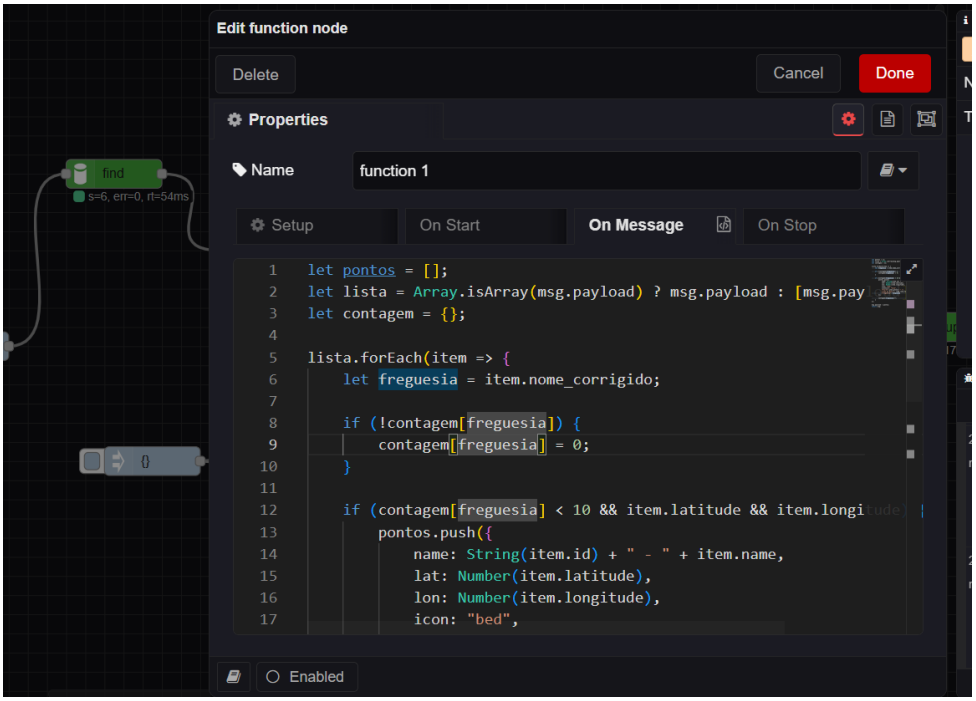


Figura 25 - Exemplo do código de um nodo function

Agora com estes dados extraídos da MongoDB, seguiu-se para a parte final deste projeto: a representação interativa dos alojamentos no mapa de Portugal.

Seguiu-se da seguinte maneira: Foi utilizado um nodo worldmap, este atua como um servidor web *self-hosted* que recorre a bibliotecas de mapeamento de código aberto, como o Leaflet e dados do OpenStreetMap, para *renderizar* um painel interativo em outra janela do explorador web/browser.

Assim que o nó *worldmap* recebe os dados formatados para a sua representação, especialmente baseado nos dados latitude e longitude falados anteriormente, desenha automaticamente os marcadores no ecrã, permitindo ao utilizador navegar no mapa de Portugal e analisar graficamente o cruzamento entre as regras climáticas, freguesias e as ofertas de alojamentos (fig. 26).



Figura 26 - Exposição dos dados no mapa interativo

No final ocorreu-me a ideia de que a informação da meteorologia era só do momento em que eu tinha gerado os ficheiros output no KNIME, e como eu queria que as informações estivessem o mais fidedignas possível, eu no Node-RED fiz este fluxo que me permite atualizar automaticamente a temperatura e a classificação da zona a cada hora.

Funciona da seguinte forma: O nodo cinzento *injetor* tem uma definição que me deixou optar por automaticamente iniciar a cada hora, ou seja, enviava um pedido novo a cada hora para iniciar este fluxo. De seguida outro nodo MongoDB com uma operação *find* iria buscar todas as freguesias, depois o nodo *split* serviu para separar as freguesias, já que isto iria atualizar todas estas. Todas as freguesias em separado foram processadas no nodo function3 que com o seu código JavaScript, selecionava os dados de *latitude* e *longitude* para criar um URL específico para cada freguesia e inserir no próximo nodo *HTTP Request* que chama outra vez a API do *openweathermap*.

Após receber a informação da meteorologia de cada freguesia, vem o nodo *function4* que contém código JavaScript, sendo este usado para aplicar as mesmas regras definidas no nodo *Rule Engine* no KNIME, para finalmente serem aplicadas pelo nodo MongoDB que tem a função de updateOne, individualmente atualizava os campos da meteorologia e resultados de cada freguesia na base de dados, para consequentemente ser atualizado no mapa interativo (fig. 27).

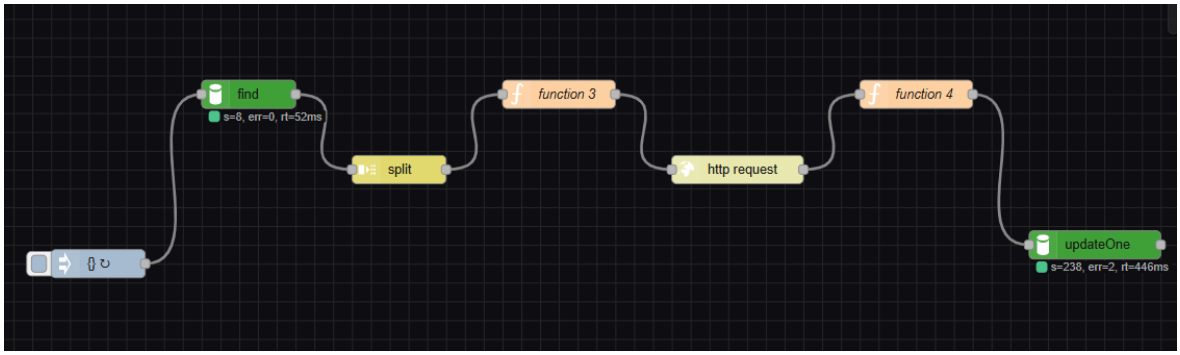


Figura 27 - Fluxo geral para atualização dos dados API

5 - Vídeo com demonstração



6 - Conclusão

Com a realização deste trabalho prático, consegui desenvolver e implementar uma solução completa de ETL aplicada ao contexto real do alojamento local em Portugal. A utilização do KNIME revelou-se fundamental para consolidar, limpar, anonimizar e formatar grandes quantidades de dados provenientes de múltiplas fontes (ficheiros CSV, regras em XML e APIs REST de câmbio e meteorologia).

Posteriormente, a persistência dos dados no MongoDB Atlas e a orquestração de fluxos em Node-RED (implementado via Docker) permitiram não só criar uma visualização interativa dos dados através do nó *worldmap*, mas também garantir a atualização autónoma e periódica das condições meteorológicas e classificações de cada freguesia a todas as horas.

Em geral, o projeto cumpriu todos os objetivos propostos, demonstrando na prática a integração entre diferentes sistemas, formatos de dados e serviços web, tal como ele contribuiu para o meu crescimento académico e sobretudo experiencia no uso de diferentes plataformas de processamento de dados, bases de dados e orquestradores de dados.

7 - Bibliografia

https://www.mongodb.com

https://frankfurter.dev/pt/

https://home.openweathermap.org/

https://insideairbnb.com/pt/get-the-data/

https://app.docker.com

https://nodered.org/docs/

https://www.knime.com